

Hybrid Search in PostgreSQL

Combining Vector and Full-Text for Real-World Applications

Shayon Sanyal · Principal PostgreSQL Specialist SA · Lead, Agentic AI for Databases

Postgres Summit US 2026 · New York City
September 30 · 10:30-11:20 EDT · Letterpress

Two questions. Two different misses.

KEYWORD SEARCH MISSES

“Where should I park my rainy-day / emergency fund?”

The best answer is **#2** for vector search. Keyword search ranks it #29, BM25 #40, and requiring every word finds nothing.

VECTOR SEARCH MISSES

“Employer rollover from 403b to 401k?”

The only answer is **#1** for BM25 and **#26** for vector search. Exact identifiers are what keyword search is for.

Words find exact terms. Vectors find paraphrases. Which one wins, and whether combining them helps, is a measurement, not a belief.



ABOUT ME

Shayon Sanyal

Principal PostgreSQL Specialist Solutions Architect

Tech Lead — Agentic AI for Databases

I help teams build on PostgreSQL, from relational applications to retrieval and agent workflows.

Today: readable SQL, measured results, and a skill you can point at your own tables.

linkedin.com/in/shayonsanyal

How we'll measure

648 questions with known right answers.

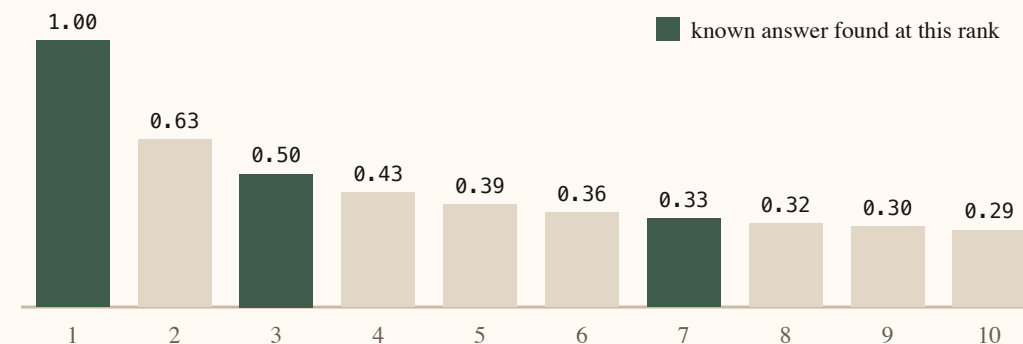
FiQA-2018: **57,638** real finance forum posts. Human judges marked which posts answer each of **648** test questions (**1,706** judgments).

NDCG@10 scores a ranking higher when those answers sit near the top.

100 means every answer is first; 0 means none is in the top 10.

Credit for a known answer at each rank

$1 / \log_2(\text{rank} + 1)$: an answer at rank 1 is worth more than three answers at rank 10



The stack: one PostgreSQL database

PIECE	VERSION	JOB
PostgreSQL	18.6	<code>tsvector</code> , <code>GIN</code> , <code>ts_rank_cd</code> , the fusion SQL, the scoring SQL
pgvector	0.8.6	<code>vector(1536)</code> , HNSW, iterative index scans, <code>halfvec</code> , <code>bit</code>
pg_textsearch	1.4.0	BM25 index and <code><@></code> operator (PostgreSQL license)
Cohere Embed v4 · Amazon Bedrock	1536 dims	Posts as <code>search_document</code> , questions as <code>search_query</code>
Cohere Rerank · Amazon Bedrock	3.5	Reorders a top 50 outside the database
VS Code + SQLTools		Numbered SQL files you can run yourself

Everything except the two model calls is SQL. Every number today is from this laptop.

One row, three representations • 01_schema.sql

```
CREATE TABLE docs (  
  id          text PRIMARY KEY,  
  body        text NOT NULL,  
  tsv         tsvector GENERATED ALWAYS AS (to_tsvector('english', body)) STORED,  
  embedding   vector(1536)      -- Cohere Embed v4, input_type = 'search_document'  
);  
CREATE INDEX ON docs USING gin (tsv); -- 0.6 s  
CREATE INDEX ON docs USING hnsw (embedding vector_cosine_ops); -- 30.4 s  
CREATE INDEX ON docs USING bm25 (body) WITH (text_config = 'english'); -- 3.9 s
```

The lexemes are **generated** and cannot drift from the text. The embedding is not: re-embed when the text or the model changes.

Pitfall 1: every word must match · 03

```
SELECT websearch_to_tsquery('english', 'Where should I park my rainy-day / emergency fund?');  
-- 'park' & 'rainy-day' <-> 'raini' <-> 'day' & 'emerg' & 'fund'
```

`websearch_to_tsquery` and `plainto_tsquery` join words with **AND**. A natural question rarely has all its words in one answer.

405 of 648 questions match no post at all · NDCG@10 4.3

Match any word, rank the matches • 04

```
WITH question AS (  
  SELECT replace(plainto_tsquery('english', body)::text, ' & ', ' | ')::tsquery AS tsq  
  FROM active_query  
)  
SELECT d.id, ts_rank_cd(d.tsv, q.tsq) AS score  
  FROM docs d, question q  
 WHERE d.tsv @@ q.tsq  
 ORDER BY score DESC  
 LIMIT 50;
```

Recall@50 doubles (6.7 → 14.6), but NDCG@10 **falls to 2.8**: with no IDF, “fund” and “day” outweigh “rainy-day”. The rainy-day question matches **10,460 posts**, and `ts_rank_cd` scores every one of them: **115 ms**.

ts_rank_cd is not BM25 · 05

```
SELECT id, -(body <@> to_bm25query(:'question', 'docs_body_bm25')) AS bm25
FROM docs
ORDER BY body <@> to_bm25query(:'question', 'docs_body_bm25')
LIMIT 50;
```

BM25 weighs rare terms up (IDF), saturates repeats, and normalizes for length. The index keeps the corpus statistics and returns the top 50 **without scoring every match**.

23.6 is in line with published BM25 results on FiQA (23.6–24.4).

KEYWORD ARM	NDCG@10	TOP 50, RAINY-DAY QUESTION
ts_rank_cd, any word	2.8	115 ms
BM25, pg_textsearch	23.6	0.56 ms

Semantic candidates • 06

```
SET hnsw.ef_search = 100;           -- default 40: LIMIT 50 would silently return 40
SELECT id, 1 - (embedding <=> (SELECT embedding FROM active_query)) AS cosine
  FROM docs
 WHERE embedding IS NOT NULL
 ORDER BY embedding <=> (SELECT embedding FROM active_query)
 LIMIT 50;
```

Keep the distance operator in `ORDER BY`, ascending, with a `LIMIT`. Embed questions as `search_query` and posts as `search_document`.

NDCG@10 **54.0** • p50 **3.9 ms**

Pitfall 2: adding scores from different scales · 07a 07b

RAINY-DAY QUESTION, EACH LIST'S TOP 50		MIN	MAX
ts_rank_cd		1.70	5.40
cosine similarity		0.478	0.598

FUSION	NDCG@10
keyword_score + cosine	5.6
keyword list, then vector list, duplicates removed	2.9
Reciprocal Rank Fusion of the same two lists	33.8

Same inputs, 6× the score. Normalizing flags make ts_rank_cd bounded, not comparable.

Fuse ranks, not scores

$$\text{RRF}(d) = \sum 1 / (60 + \text{rank}_{\text{list}}(d))$$

“TRANSFER STOCK I OWN INTO MY ROTH IRA?” (FIQA 8512)	BM25 RANK	VECTOR RANK	RRF	FUSED
The known answer	4	9	$1/64 + 1/69 = \mathbf{0.03012}$	#1
Another post	2	17	$1/62 + 1/77 = 0.02912$	#2
Another post	24	3	$1/84 + 1/63 = 0.02778$	#3
Vector's #1	—	1	$0 + 1/61 = 0.01639$	#9

Missing from a list contributes **0**. Agreement between lists wins; no score scales involved.

RRF is a full outer join • 08a

```
WITH keyword AS (  
  SELECT id, row_number() OVER (ORDER BY ts_rank_cd(tsv, q) DESC, id) AS rank  
    FROM docs, (SELECT replace(plainto_tsquery('english', :'question')::text,  
                                ' & ', ' | '))::tsquery AS q) question  
  WHERE tsv @@ q ORDER BY rank LIMIT 50  
,  
semantic AS (  
  SELECT id, row_number() OVER (ORDER BY distance, id) AS rank  
    FROM (SELECT id, embedding <=> :'question_embedding' AS distance  
          FROM docs WHERE embedding IS NOT NULL  
          ORDER BY distance LIMIT 50) nearest  
)  
SELECT coalesce(k.id, v.id) AS id,  
       coalesce(1.0 / (60 + k.rank), 0) + coalesce(1.0 / (60 + v.rank), 0) AS rrf  
  FROM keyword k FULL OUTER JOIN semantic v USING (id)  
 ORDER BY rrf DESC  
LIMIT 10;
```

One statement, two indexes (GIN and HNSW). p50 **158 ms** with `ts_rank_cd` ; **9.7 ms** with BM25 as the keyword list (08b).

Knobs that change different things

KNOB	CONTROLS	WATCH
Candidate depth (50)	How many posts each list contributes	A post outside both lists can't be fused or reranked: Recall@50
k (60)	How fast credit falls with rank	Smaller k favors each list's top few
Weights (1.0 / 1.0)	Trust in each list	FiQA, RRF · BM25: 41.3 at 1.0, 52.0 at 0.1. Never above vector alone (54.0)
hnsw.ef_search (100)	Work per HNSW scan	Must be $\geq$ the candidate depth

Change one at a time, and re-run the 648 questions.

Same question, every arm, graded

FiQA test question 9961, with 1 known answer

Employer rollover from 403b to 401k?

A in 3 of 4 top 10s

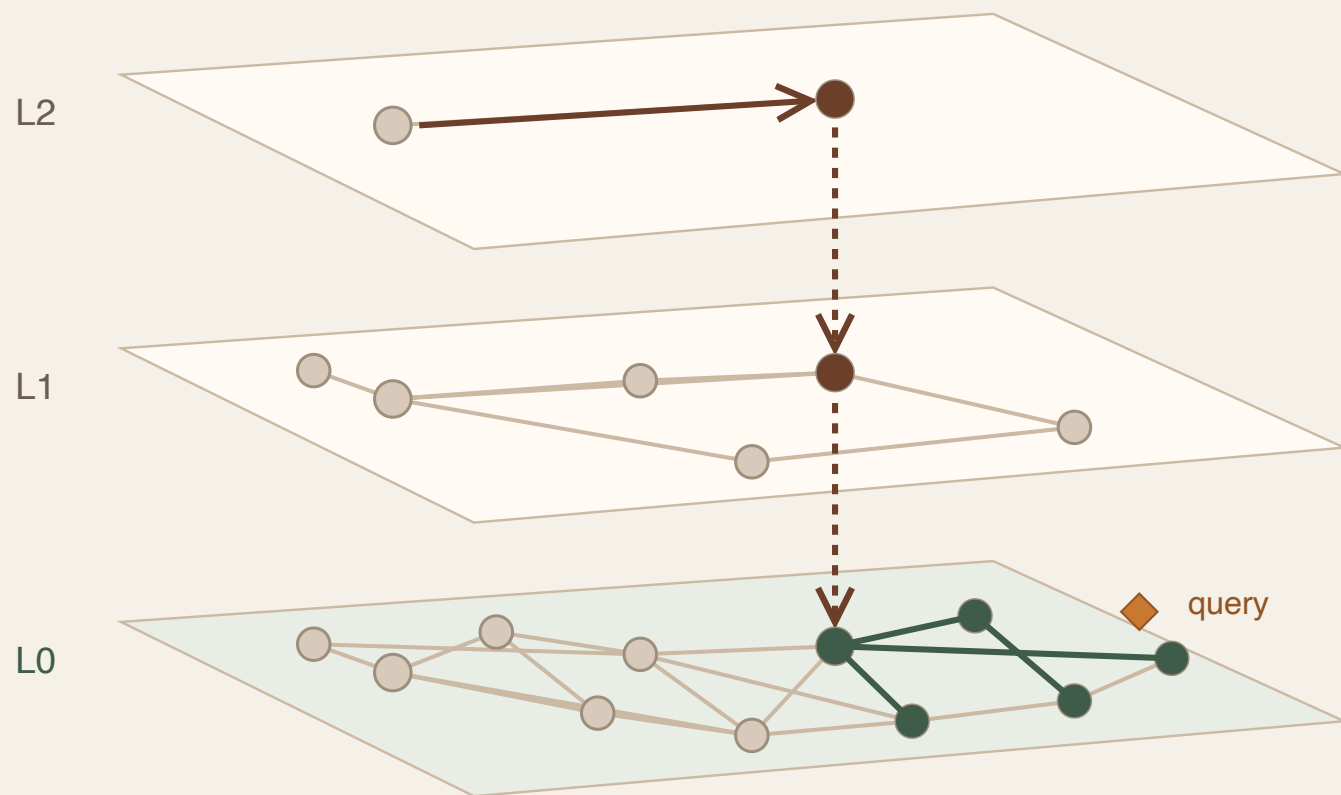
BM25 SQL 100 NDCG@10 1 of 1 answers in the top 10 12.3 ms, live SQL 1 A "403b plans are used by school districts, colleges and universities, nonprofit hospitals, charitable foundations and t... 2 "You can move money from a 403b to a 401k plan, but the question you should ask yourself is whether it is a wise... 3 I've changed jobs several times and I chose to rollover my 401k from the previous employer into an IRA instead o...	Vector SQL 0 NDCG@10 0 of 1 answers in the top 10 4.8 ms, live SQL 1 "You can move money from a 403b to a 401k plan, but the question you should ask yourself is whether it is a wise... 2 You need to check with your employer. It is called an in-service rollover and it is up to your employer on whether or not it is... 3 No that is not a rollover. Many employees have experienced a change of management companies. Sometimes...	RRF · BM25 SQL 33 NDCG@10 1 of 1 answers in the top 10 7.9 ms, live SQL 1 keyword 2 vector 1 "You can move money from a 403b to a 401k plan, but the question you should ask yourself is whether it is a wise... 2 keyword 10 vector 2 You need to check with your employer. It is called an in-service rollover and it is up to your employer on whether or not it is... 3 keyword 3 vector 11 I've changed jobs several times and I	Vector + Rerank SQL 63 NDCG@10 1 of 1 answers in the top 10 2,777.4 ms, stored run 1 stayed at 1 "You can move money from a 403b to a 401k plan, but the question you should ask yourself is whether it is a wise... 2 A up 24, was 26 "403b plans are used by school districts, colleges and universities, nonprofit hospitals, charitable foundations and t... 3 stayed at 3 No that is not a rollover. Many employees
--	--	---	---

localhost:8018 · hybrid-lab/README.md

From text to vector

- **Cohere Embed v4** on Amazon Bedrock: 1536 dimensions, unit-length vectors, so cosine distance and inner product rank the same way.
- **Two input types.** Posts are embedded as `search_document`, questions as `search_query`. Swapping them lowers relevance without any error.
- **One model per column.** Vectors from different models or versions are not comparable, even at the same dimension. Store the model id.
- **Cost of the corpus:** 7.7M words, one pass, about 35 minutes behind a tokens-per-minute quota. Resume on NULLs; wait out throttling.

HNSW: navigate broadly, then search locally



1. Find a promising region

Greedy moves across sparse links.

2. Descend at the same point

More nodes, more local detail.

3. Explore nearby candidates

A bounded search; ef controls its breadth.

Conceptual hierarchy · ef_search bounds the base-layer candidate queue

Pitfall 3: a filter is not a pre-filter • 09

```
SELECT id FROM docs
WHERE tsv @@ plainto_tsquery('english', 'mortgage')      -- 4.2% of posts
ORDER BY embedding <=> : 'question_embedding' LIMIT 10;
```

SETTING	PLAN	ROWS RETURNED
defaults (iterative_scan = off, ef_search = 40)	HNSW, then filter: Rows Removed by Filter: 38	2 of 10
SET hnsw.iterative_scan = relaxed_order	HNSW keeps walking until 10 pass	10 of 10
a rare term: heloc (0.2%)	GIN, then exact sort	10 of 10

pgvector 0.8 iterative scans are bounded by `hnsw.max_scan_tuples` (20,000).

Similar to the question AND contains a word • 11

```
SELECT h.*, left(d.body, 60)
FROM hybrid_search(:'question', :'question_embedding',
                  match_count => 5, required_terms => 'mortgage') h
JOIN docs d ON d.id = h.doc_id;
```

```
CREATE FUNCTION hybrid_search(...) ... LANGUAGE sql STABLE
SET hnsw.ef_search = 100
SET hnsw.iterative_scan = relaxed_order
SET plan_cache_mode = force_custom_plan -- keeps "required_terms IS NULL OR ..." indexable
```

The requirement goes into **both** candidate lists, before their LIMIT.

Read the plan that actually ran

```
Limit (actual rows=2)
->  Index Scan using docs_embedding_hnsw on docs (actual rows=2)
      Order By: (embedding <=> (InitPlan 1).col1)
      Filter: (tsv @@ '''mortgag'''::tsquery)
      Rows Removed by Filter: 38
```

- **Which index?** HNSW, GIN, BM25, or a sequential scan.
- **Rows removed by filter** after an approximate scan: a short result in disguise.
- **Inside functions**, use `auto_explain` with `log_nested_statements`.

Scoreboard: 648 questions

Arm	NDCG@10 (bar) and Recall@50 (line)	Recall@50	p50 ms	Beat / lost to vector
Vector	54.0	78.9	3.9	baseline
Vector · halfvec	54.0	78.9	3.9	1 / 2
Vector · binary + rescore	53.9	78.3	3.3	4 / 5
RRF · ts_rank_cd + Rerank	51.4	72.7	1079.1	163 / 228
RRF · BM25 + Rerank	50.7	75.7	1119.3	158 / 239
Vector + Rerank	50.0	78.9	424.9	160 / 242
Vector ∪ BM25 + Rerank	49.3	76.9	1036.1	150 / 251
RRF · BM25	41.3	75.7	9.7	98 / 329
RRF · ts_rank_cd	33.8	72.7	158.0	37 / 445
BM25	23.6	47.1	5.2	39 / 442
Score sum	5.6	16.0	144.7	8 / 512
Keyword · all words	4.3	6.7	0.5	6 / 506
Concatenate	2.9	14.9	154.3	6 / 518
Keyword · ts_rank_cd	2.8	14.6	174.8	6 / 518

Bar: NDCG@10 · line: Recall@50 · laptop latency, rerank rows include Bedrock calls · results/scoreboard.md

Rerank: measure it too

CANDIDATES SENT TO COHERE RERANK 3.5	NDCG@10	RECALL@50	P50
none: vector alone	54.0	78.9	3.9 ms
vector top 50	50.0	78.9	425 ms
RRF · BM25 top 50	50.7	75.7	1,119 ms
vector $\cup$ BM25, up to 100	49.3	76.9	1,036 ms

It lifts the 403b answer from #26 to #2, but puts a known answer first on 48.0% of questions. Embed v4 alone: 53.7%.

- A reranker reads the question and each candidate **together**. It can only reorder what the lists found: Recall@50 is its ceiling.
- It runs **outside PostgreSQL**: a network call per question, and the candidates' text leaves the database.

Storage: the same vectors, smaller

INDEX ON EMBEDDING	BYTES PER VECTOR	HNSW SIZE	NDCG@10	P50 MS
<code>vector(1536)</code>	6,148	450 MB	54.0	3.9
<code>halfvec(1536)</code> expression index	3,080	225 MB	54.0	3.9
<code>binary_quantize()</code> + rescore 200	200	28 MB	53.9	3.3

Half the index size for the same NDCG; 1/16 with binary plus a rescore. The table keeps full-precision vectors; only the index shrinks.

What these numbers do and don't prove

THEY SHOW	THEY DON'T SHOW
Which arm ranks FiQA's judged answers higher	That the same holds for your data
The pitfalls, reproduced with real plans	Production latency: this is one laptop, one client
Relative cost of each arm on 57,638 posts	Behavior at 50 million rows or under concurrency
This embedding and rerank model pair	Future models: they improve, and results depend on the model versions used
A public benchmark	That the models never saw similar text in training

FiQA judges are incomplete: an unjudged post can be a good answer and still count as a miss.

THREE THINGS TO TAKE HOME

**Rank words with BM25.
Fuse only what earns its place.
Measure before you ship.**

On FiQA, one vector query beat every hybrid and reranked variant.

Your data may disagree. Find out with your own questions and one SQL file.

Take it home

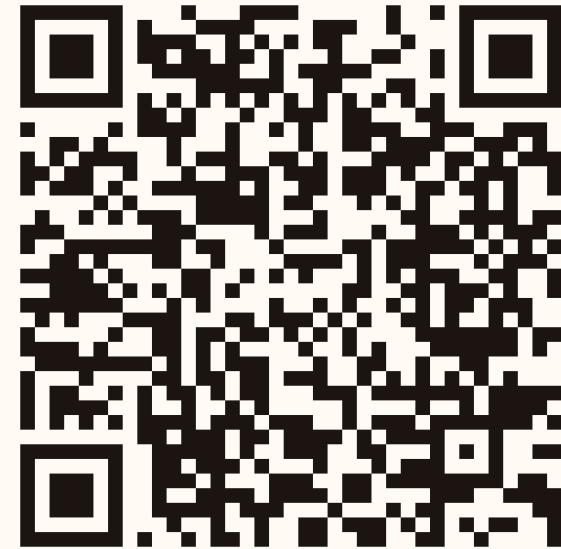
The lab: `hybrid-lab/`. Numbered SQL, VS Code cells, the UI, and the scoreboard.

The skill: adds hybrid search to *your* table and measures it.

```
/plugin marketplace add shayons/talks  
/plugin install postgres-hybrid-search@shayons-talks
```

Other agents: copy `hybrid-search-
plugin/skills/postgres-hybrid-search/`.

github.com/shayons/talks · [conferences/2026-postgresconf-agentic-ai](#)



Thank you.

Which of your queries needs both words and meaning?

Shayon Sanyal · [linkedin.com/in/shayonsanyal](https://www.linkedin.com/in/shayonsanyal)

github.com/shayons/talks

Appendix • operate the measured workload

AREA	INSPECT BEFORE CHANGING IT
HNSW build	<code>maintenance_work_mem</code> (graph must fit), parallel workers, <code>m</code> , <code>ef_construction</code> , index size
HNSW search	<code>ef_search</code> $\geq$ LIMIT, filtered recall, <code>iterative_scan</code> , <code>max_scan_tuples</code>
Full text	OR vs AND queries, match counts, <code>ts_rank_cd</code> cost on common terms, BM25 availability
Embeddings	Model id per column, input types, re-embedding on text change, quota and throttling
Functions	Inner plans with <code>auto_explain</code> ; <code>plan_cache_mode</code> for optional filters
Evaluation	Re-run the question set after every change; keep per-question wins and losses

References

- **PostgreSQL 18:** Full-text search, Using EXPLAIN, auto_explain
- **pgvector 0.8.6:** HNSW, filtering, iterative index scans · **pg_textsearch 1.4.0:** BM25 for PostgreSQL
- **RRF:** Cormack, Clarke & Büttcher, SIGIR 2009 · **HNSW:** Malkov & Yashunin, arXiv:1603.09320
- **FiQA-2018** via **BEIR** · Dave Ebbelaar, hybrid-retrieval tutorial (the FiQA + NDCG approach this lab moves into PostgreSQL)
- **This talk:** [hybrid-lab/sql/](#), [hybrid-lab/results/scoreboard.md](#), [hybrid-search-plugin/](#)

Measured on PostgreSQL 18.6, pgvector 0.8.6, pg_textsearch 1.4.0 · September 2026